Lab 5: Static Code Analysis

Pylint · Bandit · Flake8 on **parking_lot_system.py**

Field	Details
Name	AMOGH SHARMA
SRN	PES1UG24CS053
Course	Software Engineering (Fifth Semester)
Assigned repo	SETAPESU26/L5_51_parking_lot_system
Submission repo	Chunnu-Munnu/SE-LAB-PES1UG24CS053, folder `Lab-5/`
Date	14 October 2026

1. Objective

Use three static analysis tools to find and fix quality, style and security problems in the assigned script: **Pylint** (code quality and logic), **Flake8** (PEP 8 style) and **Bandit** (security). Targets from the handout: handle every Bandit issue, every Flake8 issue, and get the Pylint score above 9.5.

2. Program under test

`parking_lot_system.py` is a small parking lot manager. It parks vehicles by licence plate, frees slots, computes a surge fee, hashes a plate, issues a ticket PIN and exports a manifest file. The original version ran, but printed its secret barrier key, printed its lines out of order and relied on `eval`, `os.system` and MD5.

3. Setup and commands

```
python -m venv venv
venv\Scripts\activate            # source venv/bin/activate on Linux/macOS
pip install pylint bandit flake8

python parking_lot_system.py
pylint parking_lot_system.py      > pylint_report.txt
bandit -r parking_lot_system.py   > bandit_report.txt
flake8 parking_lot_system.py      > flake8_report.txt
```

Tool versions: Pylint 4.1.3, Bandit 1.9.4, Flake8 7.4.1 on Python 3.14.0.

4. Results: before vs after

Tool	Before	After
Pylint	4.10 / 10 (36 messages)	**10.00 / 10** (0 messages)
Bandit	5 issues (2 High, 1 Medium, 2 Low)	**0 issues**
Flake8	10 issues	**0 issues**

Full reports are in `Lab-5/Reports/Before/` and `Lab-5/Reports/After/`. An empty `flake8_report.txt` means Flake8 found nothing.

5. Issue table

Line numbers refer to the original file. High and medium severity issues are listed first.

#	Issue	Tool / Code	Severity	Type	Line(s)	Description	Fix Approach
1	Shell command injection	Bandit B605	High	Security	64	`os.system("echo ... " + filename)` runs a shell with the filename pasted in, so a name like `x.txt & del *` would run extra commands.	Removed the shell call and print the message with `print()`.
2	Weak MD5 hash	Bandit B324	High	Security	53	MD5 is broken for collision resistance and should not be used to fingerprint plates.	Switched to `hashlib.sha256`.
3	Hardcoded secret printed to console	Manual review (CWE-798)	High	Security	76, 85	`gate_barrier_key = "BARRIER_GATE_SECRET_TOKEN_2026"` is committed in source and printed in plain text on every run. Bandit misses it because the variable name doesn't match its password regex.	Read the key from the `GATE_BARRIER_KEY` environment variable and only print whether it is configured.
4	Use of `eval()`	Bandit B307, Pylint W0123	Medium	Security	46–49	`calculateParkingFee` evaluates a caller-supplied string, which allows arbitrary code execution.	Function now takes a numeric `surge_multiplier` and computes `rate * float(surge_multiplier)`.
5	Mutable default argument	Pylint W0102	Medium	Bug	15	`entry_logs=[]` is created once and shared across every call, so logs leak between calls.	Default changed to `None`; a new list is created inside the function.
6	Try / except / pass with bare `except`	Bandit B110, Pylint W0702, Flake8 E722	Low (Bandit)	Bug	28–33	`releaseSlot` silently swallows every exception (even typos), and the caller can't tell whether the release worked.	Look the slot up explicitly, print a message and return `False` for unknown slots, `True` on success.
7	Insecure random for ticket PIN	Bandit B311	Low	Security	57	`random.randint` is a predictable PRNG; ticket PINs could be guessed.	Use `secrets.randbelow(9000) + 1000`.
8	`global` statement	Pylint W0603	Warning	Design	16	`parkVehicle` rebinds the module-level `total_collections` through `global`.	Keep the running total in a module-level `ledger` dict that the function updates.
9	File opened without `with` / encoding	Pylint R1732, W1514	Warning	Resource leak	61–63	The file handle is not closed if `write()` fails, and the encoding depends on the OS locale.	`with open(filename, "w", encoding="utf-8") as manifest:`
10	Parameter shadows a global	Pylint W0621	Warning	Bug risk	67	`getAvailableSummary(parking_slots)` hides the module-level `parking_slots`.	Parameter renamed to `slots`.
11	Unused variables	Pylint W0612, Flake8 F841	Warning	Code smell	37, 47, 69	`overtime_fee`, `base_rate` (only used inside the `eval` string) and loop variable `s` are never used.	Removed them; the loop iterates `slots.values()`.
12	Unused import	Pylint W0611, Flake8 F401	Warning	Code smell	4	`math` is imported but never used.	Removed the import (`random` was also dropped when the PIN moved to `secrets`).
13	Comparison to `None` with `==`	Flake8 E711, Pylint C0121	Convention	Style	38	`parking_slots.get(slot_id) == None`	`is None`
14	Simplifiable if / unnecessary else	Pylint R1703, R1705	Refactor	Style	40–43	`if x: return True else: return False`	`return not slot["is_occupied"]`
15	Lines too long	Flake8 E501, Pylint C0301	Convention	Style	7, 8, 9, 23	Lines of 82–131 characters (limit 79 for Flake8).	Wrapped the dict entries and the log message.
16	`%` string formatting	Pylint C0209	Convention	Style	23	Old-style `%` formatting.	f-string.
17	camelCase function names	Pylint C0103	Convention	Style	15, 27, 36, 46, 52, 56, 60, 67	`parkVehicle`, `releaseSlot`, … break PEP 8 naming.	Renamed to `park_vehicle`, `release_slot`, `check_slot_availability`, `calculate_parking_fee`, `generate_vehicle_hash`, `generate_ticket_pin`, `export_parking_manifest`, `get_available_summary`.

#	Issue	Tool / Code	Severity	Type	Line(s)	Description	Fix Approach
18	Constant naming	Pylint C0103	Convention	Style	12	`total_collections` treated as a constant but mutated.	Replaced by the `ledger` dict (see #8).
19	Missing docstrings	Pylint C0114, C0116	Convention	Documentation	1, 15, 27, 36, 46, 52, 56, 60, 67, 75	No module or function docstrings.	Added a module docstring and one per function.
20	Missing final newline	Flake8 W292, Pylint C0304	Convention	Style	88	File ends without `\n`.	Added the newline.

6. Problems the tools did not flag

Found by running the script and reading the output, as the assignment README suggests.

#	Problem	Line(s)	What was wrong	Fix
A	Output printed out of order	64	The `os.system` subprocess writes straight to the console while Python's own `print` output is still buffered, so "Parking manifest exported…" appeared as the **first** line of output.	Fixed by #1: the message now goes through `print()` and appears in order.
B	Secret token shown in output	85	`Barrier Key active: BARRIER_GATE_SECRET_TOKEN_2026` was printed on every run.	Fixed by #3.
C	Log line never shows the charge	23	The entry log said "with total charge calculated" but never included the amount, and `%d` would truncate fractional hours.	Log now reads e.g. `... for 3 hour(s), charge Rs 60.00`.
D	Zero or negative bookings accepted	15–24	`parkVehicle("P02", "X", -2)` succeeded and reduced `total_collections`.	`park_vehicle` rejects `hours_booked <= 0`.
E	Script runs on import	88	`main()` was called at module level, so importing the module ran the demo and wrote a file.	Wrapped in `if __name__ == "__main__":`.

7. Commits

One baseline commit, then one commit per tool:

1. `Lab 5: add assigned parking_lot_system.py and baseline Pylint/Bandit/Flake8 reports`
2. `Lab 5: fix Bandit security issues (eval, MD5, random PIN, os.system, try/except/pass, hardcoded key)`
3. `Lab 5: fix Flake8 issues (unused import/variable, long lines, None comparison, final newline)`
4. `Lab 5: fix Pylint issues (10.00/10) and add issue table, reflection and report`

8. Reflection

1. Which issues were the easiest to fix, and which were the hardest? Why?

The easiest were the Flake8 style issues: the unused `math` import, `== None` → `is None`, the missing final newline and the long lines. Each one points at an exact line and the fix is mechanical, with no effect on behaviour. Swapping MD5 for SHA-256 and `random.randint` for `secrets.randbelow` was just as quick.

The hardest was the `eval()` in `calculateParkingFee`. Deleting it was not enough, because the caller was passing a whole expression (`"base_rate * 1.50"`) as a string. I had to change the function's contract to accept a numeric surge multiplier and update the call site, while making sure the fee for P03 still came out as 45.0. Removing the `global` statement and the mutable default argument also took some thought, since both change how state moves through the program: the log list is now passed in from `main()` and the running total lives in a `ledger` dict. The hardcoded barrier key was tricky in a different way. Bandit did not report it at all, so I only found it by reading the code and the output.

2. Did the static analysis tools report any false positives? If so, describe one example.

Mostly the findings were real, but a few were debatable:

- **Pylint C0103 on** `total_collections` asked for an UPPER_CASE name because it is a module-level variable. It is not a constant, though; it is a running total that changes every time a car parks. Renaming it to `TOTAL_COLLECTIONS` would have silenced the warning and made the code more misleading. The real problem was the global mutable state, so I moved the total into a `ledger` dict instead of just renaming it.
- **Bandit B324 (MD5)** could be a false positive in a different program. If the hash were only used as a non-secret lookup key, `hashlib.md5(..., usedforsecurity=False)` would be acceptable. Here the plate hash is used as a fingerprint, so switching to SHA-256 was the safer choice.

I also saw the opposite problem: Bandit **missed** the hardcoded `BARRIER_GATE_SECRET_TOKEN_2026` because its B105 check matches variable names like `password` or `token`, and this one was called `gate_barrier_key`. A clean tool report doesn't mean the code is secure.

3. How would you integrate static analysis tools into your actual software development workflow?

- **Locally:** a `pre-commit` hook that runs Flake8 and Bandit on staged files, plus editor integration (VS Code's Pylint/Flake8 extensions) so problems show up while typing instead of after the fact.
- **In CI:** a GitHub Actions job on every push and pull request that runs `pylint --fail-under=9.5`, `flake8` and `bandit -r . -ll` (fail on medium and high). A PR can't be merged while this check is red.
- **Shared config:** keep the rules in `setup.cfg` / `pyproject.toml` / `.bandit` in the repo so everyone runs the same checks, and only suppress a warning with an inline `# nosec` or `# pylint: disable=` plus a comment explaining why.
- **Gradual adoption on old code:** run the tools in report-only mode first, fix high-severity security findings immediately, then raise the Pylint threshold over time.

4. What tangible improvements did you observe in the code quality, readability, or potential robustness after applying the fixes?

- **Security:** the five Bandit findings (2 High, 1 Medium, 2 Low) went to zero. There is no more `eval`, no shell call built from a filename, no weak hash or predictable PIN, and the barrier key is no longer stored in source or printed.
- **Correctness:** the shared-list bug from `entry_logs=[]` is gone, unknown slots are reported instead of silently ignored, zero or negative bookings are rejected, and the manifest file is always closed. The output is also in the right order now; before, the `os.system` line printed first.
- **Readability:** Pylint went from **4.10/10 to 10.00/10**. Every function has a docstring and a snake_case name, `check_slot_availability` is two lines instead of six, and `get_available_summary` is a single `sum(...)`.

- **Maintainability:** the module can be imported without side effects thanks to the `__main__` guard, so the functions can be reused or unit-tested. Flake8 reports nothing, so future diffs won't be cluttered with style noise.